

1) Audit (Days 1–2): Stabilize, measure, and make risk visible

Outcomes by end of Audit

- **Crash causes identified** (top 3) + immediate mitigations
- **Baseline metrics** captured (latency, throughput, error rate, cost/doc, drift)
- **Safety rails** in place: retries, circuit breakers, timeouts, dead-letter queue (DLQ)
- **Current behavior “frozen”** via goldens (sample docs + expected labels)

What you do

A. Create an “Observation Patch” PR (no refactor yet)

Add:

- Structured logging (JSON), correlation IDs per document/batch
- Metrics: docs_processed, classification_latency_ms, llm_calls, timeouts, parse_failures, provider_errors, token_usage
- Central exception boundary: *no uncaught exceptions leave the pipeline*

B. Capture representative traffic

- Sample **1–2 weeks** of docs (or hashes + extracted features if storage sensitive)
- Build a stratified dataset: regulation type, language, length, source channel, known failure classes

C. Identify weekly crash patterns

- Look for: memory leaks, concurrent futures misuse, provider rate-limit handling, malformed PDF/HTML, prompt formatting exceptions, schema/JSON parsing failures

D. Establish SLOs

Given \$50k/hour downtime and regulatory penalties:

- Availability target: **99.95%+**
- Weekly crash target: **0**
- Classification latency: define p50/p95, and acceptable max for spikes
- Accuracy target: maintain or improve existing acceptance thresholds (use goldens)

Deliverable: “Audit Report” (1–2 pages) with top risks, baseline numbers, and the refactor plan scope.

2) Map (Days 2–4): Reverse-engineer the system into a clear flow

Outcomes by end of Map

- A **pipeline diagram** (ingest → extract → segment → classify → validate → persist)
- A **dependency map** of modules/functions in the 2,000-line file
- A **contract/spec** for inputs/outputs at each stage
- A **behavioral spec** for the current classifier (golden tests)

What you do

A. Break the legacy script into “stages” (even if still in one file)

Tag functions by stage and draw dataflow. Typical doc classification pipeline:

1. Ingestion (queue, batcher)
2. Extraction (PDF/text/HTML)
3. Cleaning & normalization
4. Chunking (token-aware)

5. LLM classification (prompt + model call)
6. Post-processing (structured output parsing, confidence)
7. Validation (schema + business rules)
8. Routing/storage (DB, downstream systems)
9. Observability + retry/DLQ

B. Define explicit contracts

For each stage, define:

- Input type, output type
- Error types (recoverable vs fatal)
- Performance constraints (max time, max size)
- Determinism requirements (replayability)

C. Build a Golden Dataset

- 500–2000 docs: include known tricky cases (tables, scanned PDFs, multi-language)
- Store expected outputs from current system (even if imperfect) as a temporary truth so you can refactor without breaking business flows

Deliverable: “System Map” doc: diagrams + interface contracts + golden dataset definition.

3) Modularize (Days 4–10): Build a maintainable LangChain architecture

Goals

- Prompt edits take **minutes**, not 6 hours

- Replace tight coupling with **interfaces** + dependency injection
- Make stages independently testable and swappable
- Introduce LangChain in a way that improves structure (not magic)

Target architecture (clean, layered)

Core domain

- Document, Chunk, ClassificationResult, ValidationReport

Ports (interfaces)

- ExtractorPort (pdf/html/plain)
- ChunkerPort
- ClassifierPort (LLM-based)
- ValidatorPort (schema + business rules)
- RepositoryPort (save results)
- QueuePort (consume/ack/retry/DLQ)
- TelemetryPort (logs/metrics/traces)
- PromptStorePort (versioned prompts)

Adapters (implementations)

- Extractors: pdf parser, OCR fallback (optional), html cleaner
- Classifier: LangChain chain (structured output)
- Repo: DB writer, S3 writer, etc.
- Queue: Kafka/SQS/Rabbit, etc.

Orchestration

- PipelineRunner ties ports together

LangChain implementation approach (practical)

Use LangChain for **the LLM + prompt + structured output + retries**, not for everything.

Key patterns

- **LCEL** (LangChain Expression Language) chain for classification
- **Structured outputs** enforced with schema (Pydantic/Zod style) to avoid JSON parsing crashes
- **Prompt versioning** with a “prompt registry” (DB/Git) and a rollout mechanism

Example modular flow (conceptual)

1. text = extractor.extract(doc_ref)
2. chunks = chunker.chunk(text)
3. result = classifier.classify(chunks, metadata)
4. report = validator.validate(result)
5. repository.save(doc_id, result, report)

Non-negotiable resilience rules

- Every stage has:
 - Timeout
 - Retry policy with exponential backoff
 - Circuit breaker when provider degrades
 - DLQ for poison documents
- Idempotent processing keyed by doc_id + version

Deliverable: repo structure

```
/src
  /domain
  /ports
  /adapters
  /pipeline
  /prompts
/tests
  /unit
  /integration
  /e2e
```

Deliverable: “Modular Design Spec” + initial working pipeline that mirrors legacy behavior using goldens.

4) Test (Days 7–14, overlaps with Modularize): 80%+ coverage with realistic guarantees

Coverage target

- **80%+ line coverage** overall is good, but the real goal is:
 - 95%+ for pure logic (chunking, validation, routing)
 - Strong integration coverage for LLM boundaries using mocks/record-replay

Test strategy pyramid

A. Unit tests (fast, deterministic)

- Chunking behavior (token limits, boundary conditions)
- Cleaning/normalization
- Validation rules
- Retry/backoff logic

- Prompt rendering (variables present, no formatting crashes)

B. Contract tests

- Ensure adapters obey ports:
 - Extractor returns UTF-8 text + metadata
 - Classifier returns schema-valid ClassificationResult
 - Repository idempotency and error handling

C. Integration tests (mock LLM)

- Use a fake LLM or recorded responses:
 - “Golden prompt → expected structured output”
 - Validate parsing + post-processing doesn’t regress

D. E2E tests (staging)

- Run pipeline against a sample batch and assert:
 - throughput within thresholds
 - error rate within thresholds
 - outputs match golden distribution

LLM testing best practices (what stops the weekly crashes)

- **Schema-first outputs:** if the model returns invalid JSON, you get a controlled failure + retry + DLQ, not a crash
- **Prompt snapshot tests:** changing prompts triggers a diff + review
- **Deterministic mode for tests:** temperature 0 + fixed seed if supported
- **Evaluation harness:** compare new vs old on golden dataset (accuracy + drift)

Deliverable: CI pipeline: lint + unit + integration + coverage gate at 80%.

5) Deploy (Days 10–18): Zero-downtime rollout with safety + rollback

You need to ship without another outage. Use progressive delivery.

Zero-downtime strategy (recommended)

A. Blue/Green or Canary (preferred)

- Stand up **New Pipeline (v2)** alongside **Legacy (v1)**
- Route a small % of traffic to v2:
 - Start at 1%, then 5%, 20%, 50%, 100%
- Automated rollback triggers on:
 - error rate spike
 - latency spike
 - schema validation failures
 - business KPI drift (label distribution / rejection rate)

B. Shadow mode first (best for accuracy + safety)

- v2 processes docs *in parallel* but does not write authoritative results
- Compare v2 vs v1 outputs on sampled stream
- Only promote to “active” after passing thresholds

Deployment controls

- Feature flags:

- use_v2_classifier
 - use_structured_output
 - use_new_chunker
- Prompt versions are separately deployable:
 - Prompt registry supports “draft → canary → stable”

Data migration (if needed)

- Write results with version tag: classifier_version, prompt_version
- Ensure idempotency to prevent double-processing charges

Observability dashboards (exec-friendly)

- Availability, throughput, p95 latency
- Weekly crash count (should be 0)
- DLQ volume + top causes
- Cost per document (token usage)
- Output drift vs baseline

Deliverable: “Release Plan” with rollback and SLO gates.

Execution plan to deliver by Friday (what you hand the exec team)

Friday deliverables (executive-ready)

1. **Refactor Blueprint** (this plan) + system diagram

2. **Risk register** with mitigations and owners
3. **Phased timeline** (2–3 week engineering plan)
4. **Testing strategy** + coverage gates
5. **Zero-downtime deployment plan** + rollback triggers

Suggested timeline (compressed but realistic)

- **Week 1:** Audit + Map + observation patch + golden dataset
 - **Week 2:** Modularize core pipeline + LangChain classifier + unit/integration tests
 - **Week 3:** Shadow + canary rollout + optimize + deprecate legacy
-

What I'd emphasize to the CTO (to get immediate buy-in)

- The weekly crash issue is usually **unhandled LLM/parse/provider failures + tight coupling**. Schema-first outputs + stage isolation will remove the “single exception kills the system” failure mode.
- Shadow/canary rollout prevents another \$2M penalty event.
- Prompt changes become safe and fast via **prompt registry + snapshot tests + canary**.